

Obiettivi e Concetti Fondamentali

La sicurezza informatica si prefigge di preservare la triade CIA: **Confidenzialità** (protezione dalla divulgazione non autorizzata), **Integrità** (protezione da alterazioni non autorizzate) e **Disponibilità** (garanzia di accesso continuo alle risorse).

Obiettivi della Sicurezza (CIA e IAAA)

- **Confidenzialità:** Garantisce che le informazioni non siano divulgate a persone non autorizzate e che l'individuo possa controllare chi gestisce i propri dati.
- **Integrità:** Assicura che i dati non vengano alterati se non da persone autorizzate e tramite protocolli specifici.
- **Disponibilità (Accessibilità):** Garantisce la continuità del servizio e l'accesso alle informazioni per chi ne ha diritto. Un esempio emblematico è il caso CrowdStrike del 2024: un aggiornamento software difettoso di un componente di sicurezza ha causato il crash di massa di sistemi Windows, rendendo indisponibili infrastrutture critiche. Le conseguenze sono state immediate e concrete: banche, ospedali, compagnie aeree e servizi essenziali si sono trovati bloccati, non per un attacco esterno, ma per un problema interno di aggiornamento.
- **IAAA:**
 - Identificazione: Fase iniziale di associazione dell'identità (es. login) eseguita una sola volta.
 - Autenticazione: Verifica ricorrente (eseguita ogni volta) che l'utente sia chi dice di essere.
 - Autorizzazione: Concessione dei permessi per attività specifiche dopo l'autenticazione.
 - Accountability: Capacità di monitorare e dimostrare le attività di un utente tramite i file di Log.

L'architettura di sicurezza per OSI (**Architettura OSI X.800**) si concentra su tre aspetti: attacchi, servizi e meccanismi.

Attacchi

1. **Attacchi Passivi:** Mirano a intercettare informazioni senza alterare le risorse. Includono il recupero del contenuto (es. leggere una mail) e l'analisi del traffico (osservare lunghezza e frequenza dei messaggi per intuire la natura della comunicazione). Sono difficili da rilevare perché non lasciano tracce, quindi si combattono con la prevenzione (crittografia).
2. **Attacchi Attivi:** Comportano la modifica dei dati o la creazione di flussi falsi. Si dividono in:
 - **Masquerade:** Un'entità finge di essere un'altra.
 - **Replay:** Cattura passiva di dati e successiva ritrasmissione per ottenere effetti non autorizzati.
 - **Modifica dei messaggi:** Alterazione, ritardo o riordino dei messaggi legittimi.
 - **Denial of Service (DoS):** Inibizione del normale utilizzo delle strutture di comunicazione.

Servizi e Meccanismi

1. **Servizi di sicurezza:** implementano le politiche di protezione (es. controllo degli accessi, non ripudio, integrità dei dati).
2. **Meccanismi di sicurezza:** mezzi tecnici per garantire i servizi, suddivisi in specifici (cifatura, firma digitale, traffic padding) e pervasivi (etichette di sicurezza, tracce di controllo, recovery).

Il Modello di Sicurezza della Rete

Per proteggere i messaggi scambiati tra due parti (**principals**) attraverso un canale logico, sono necessari due componenti:

- Una trasformazione legata alla sicurezza (es. crittografia).
- Informazioni segrete condivise (chiavi crittografiche). La progettazione richiede un algoritmo, metodi di distribuzione delle chiavi e un protocollo specifico.

Firme Digitali e Crittografia

Viene introdotta la distinzione tra cifatura simmetrica (veloce, stessa chiave per entrambi) e asimmetrica (coppia di chiavi pubblica/privata).

Crittografia: Simmetrica vs Asimmetrica

- **Cifatura Simmetrica:** È caratterizzata dall'essere più veloce. Tuttavia, richiede che le due parti si scambino la stessa chiave segreta prima di poter iniziare la comunicazione sicura. La forza di questo sistema risiede interamente nella tecnica di distribuzione delle chiavi.

- **Cifratura Asimmetrica:** Ogni utente possiede una coppia di chiavi: una pubblica (nota a tutti) e una privata (segreta e personale). Se il mittente vuole inviare un messaggio, lo cifra con la chiave pubblica del destinatario; solo quest'ultimo, usando la propria chiave privata, potrà decifrarlo.

La **firma digitale** è un meccanismo fondamentale per garantire:

1. **Autenticità:** certezza dell'identità del firmatario.
2. **Integrità:** il messaggio non è stato alterato.
3. **Non ripudio:** il firmatario non può negare l'invio.

Il processo si avvale di un **Ente Certificatore** che rilascia certificati che associano l'identità di un utente alla sua chiave pubblica. Per verificare una firma, il destinatario **confronta l'hash del messaggio ricevuto con quello decifrato tramite la chiave pubblica del mittente**. Un esempio avanzato di questo processo è l'algoritmo **RSA-PSS**, che utilizza un valore "salt" variabile per rendere ogni firma unica, anche per lo stesso messaggio.

Il Ruolo dell'Ente Certificatore

Per evitare che qualcuno distribuisca chiavi pubbliche false, interviene l'Ente Certificatore. Questo emette un certificato, un atto che attesta il legame tra l'identità di un utente e la sua chiave pubblica, firmandolo con la propria chiave privata per garantirne l'immutabilità.

Mezzi di Autenticazione dell'Utente

L'autenticazione è il **processo di verifica di un'identità rivendicata** e si divide in due fasi: **Identificazione** (presentazione di un ID) e **Verifica** (conferma del legame tramite informazioni di autenticazione). I quattro metodi generali sono:

- Ciò che l'individuo **conosce**: password, PIN o risposte a domande segrete.
- Ciò che l'individuo **possiede**: chiavi fisiche, smart card o chiavi crittografiche.
- Ciò che l'individuo **è** (biometria statica): impronte digitali, retina o volto.
- Ciò che l'individuo **fa** (biometria dinamica): pattern vocale, ritmo di battitura o firma autografa.

Distribuzione delle Chiavi Simmetriche

La forza della crittografia simmetrica risiede nella tecnica di distribuzione delle chiavi. Esistono quattro opzioni:

1. Consegna fisica da A verso B.
2. Una terza parte consegna fisicamente la chiave ad A e B.
3. Uso di una vecchia chiave per cifrare una nuova (se una chiave è compromessa, lo sono tutte le successive).
4. Utilizzo di un KDC (Key Distribution Center) che gestisce due tipi di chiavi: chiave di sessione (monouso per una singola connessione) e chiave permanente (usata tra entità e KDC per scambiare in sicurezza le chiavi di sessione).

Protocollo Kerberos: Evoluzione e Architettura

Kerberos è un servizio di autenticazione centralizzato basato su crittografia simmetrica.

Kerberos Versione 4

Il protocollo si è evoluto attraverso tre scenari di dialogo:

1. **Dialogo Semplice:** Coinvolge un AS (Authentication Server) che emette un Ticket cifrato con la chiave del server (K_s). Il ticket include l'indirizzo di rete del client (AD_c) per evitare che venga riutilizzato da un'altra postazione.
2. **Dialogo più Sicuro (TGS):** Introduce il TGS (Ticket-Granting Server) per evitare che l'utente inserisca la password per ogni servizio. L'AS rilascia un Ticket_tgs cifrato con la password dell'utente (K_c); una volta decifrato, il client usa questo ticket per richiedere al TGS i ticket specifici per i singoli servizi (Ticket_v).
3. **Architettura Definitiva (Autenticatore):** Per contrastare i replay attack, viene introdotto l'Autenticatore. Questo contiene l'ID e il Timestamp del client, cifrati con la chiave di sessione ($K_{c,tgs}$). Avendo durata brevissima, impedisce a un attaccante di riutilizzare un ticket intercettato. Il server può fornire Autenticazione Mutua restituendo il timestamp incrementato di 1 ($TS + 1$).

Concetti Chiave di Kerberos

- **Realm (Regno):** Insieme di nodi (client e server) che condividono lo stesso database Kerberos.

- **Principal:** Nome identificativo di un utente o servizio nel sistema (composto da nome, istanza e regno).

- **Validità:** La chiave di sessione dell'AS (*LA*) deve avere una durata maggiore o uguale a quella del TGS (*LS*) per evitare attacchi a cascata su chiavi scadute.

Kerberos Versione 5

Nata per superare i limiti della V4, introduce miglioramenti in due aree:

- **Carenze Ambientali:** Indipendenza dal sistema di crittografia (non solo DES), dal protocollo di rete (non solo IP), supporto per l'ordinamento dei byte ASN.1, durata dei ticket arbitraria e possibilità di inoltrare delle credenziali.

- **Carenze Tecniche:** Eliminazione della doppia cifratura non necessaria, uso dello standard CBC (invece di PCBC), introduzione di chiavi di sottoespona e meccanismi di pre-autenticazione per ostacolare gli attacchi alle password.

Gestione delle Chiavi Pubbliche

L'uso della **crittografia asimmetrica** introduce il rischio di **chiavi pubbliche false distribuite** da un malintenzionato. La soluzione è il **Certificato a chiave pubblica**:

- Un certificato contiene la chiave pubblica e l'ID del proprietario, ed è firmato da una CA (Autorità di Certificazione) fidata.

- Lo standard universale per questi certificati è X.509, utilizzato in SSL/TLS e IPsec.

- I certificati hanno un periodo di validità e possono essere revocati (tramite CRL) se la chiave è compromessa o l'utente non è più certificato.

Infrastruttura a Chiave Pubblica (PKI)

La PKI (o PKIX nel modello X.509) è l'insieme di hardware, software e procedure per gestire i certificati. I componenti principali sono:

- End Entity: L'utente finale o dispositivo.

- CA (Certificate Authority): L'ente che emette i certificati.

- RA (Registration Authority): Svolge funzioni amministrative per conto della CA.

- Repository e CRL Issuer: Per l'archiviazione e la gestione della revoca.

Le funzioni di gestione includono la registrazione, l'inizializzazione, la certificazione e l'aggiornamento/recupero delle coppie di chiavi.

Distribuzione delle Chiavi e Certificati X.509

La crittografia a chiave pubblica presenta un punto debole: un utente potrebbe inviare una chiave pubblica falsa fingendosi qualcun altro. Per risolvere questo problema si utilizzano i Certificati a chiave pubblica:

- **Definizione:** Un certificato contiene la chiave pubblica e l'identificativo del proprietario, il tutto firmato da una CA (Autorità di Certificazione) fidata.

- **Standard X.509:** È lo schema universale per la formattazione dei certificati, utilizzato in protocolli come IPsec, SSL/TLS e S/MIME. Ogni certificato include un periodo di validità e può essere revocato prima della scadenza se la chiave è compromessa o l'utente non è più certificato (tramite le CRL - Certificate Revocation Lists).

- **Distribuzione di chiavi segrete:** I certificati possono essere usati per scambiare una chiave di sessione monouso. Il mittente cifra il messaggio con crittografia simmetrica e poi cifra la chiave di sessione con la chiave pubblica del destinatario (ottenuta dal certificato).

Dettagli Tecnici del Certificato X.509

Oltre alla struttura generale, l'analisi di un certificato tramite un "Certificate Viewer" permette di identificare parametri specifici e impronte digitali fondamentali per la sicurezza.

1. **Parametri Identificativi del Certificato:** Un certificato X.509 contiene campi standardizzati che ne definiscono l'identità e la validità:

- **Common Name (CN):** Indica il dominio specifico per cui il certificato è stato emesso (ad esempio, emailri.com).

- **Organization (O):** Identifica l'Autorità di Certificazione (CA) che ha emesso e firmato il certificato (ad esempio, Let's Encrypt).

- **Periodo di Validità:** Definisce l'intervallo temporale di efficacia del certificato attraverso due date specifiche: Issued On (data di emissione) ed Expires On (data di scadenza).

2. **SHA-256 Fingerprints** (Impronte Digitali) Le impronte digitali (fingerprint) sono hash univoci utilizzati per verificare rapidamente l'autenticità e l'integrità dei materiali crittografici senza dover analizzare l'intero file. È fondamentale distinguere tra:

- **Fingerprint del Certificato:** È l'hash SHA-256 calcolato sull'intero certificato completo (comprensivo di metadati e firma della CA).
- **Fingerprint della Chiave Pubblica:** È l'hash SHA-256 calcolato esclusivamente sulla chiave pubblica contenuta nel certificato.

Questi strumenti servono a garantire che il certificato non sia stato manipolato (integrità) e che appartenga effettivamente al soggetto dichiarato (autenticità).

Infrastruttura a Chiave Pubblica (PKIX)

Il modello PKIX (Public-Key Infrastructure con X.509) definisce l'insieme di hardware, software e procedure per gestire i certificati. I componenti principali sono:

- End Entity: L'utente finale o dispositivo.
- CA (Certificate Authority): L'ente che emette i certificati.
- RA (Registration Authority): Svolge funzioni amministrative per conto della CA.
- CRL Issuer e Repository: Responsabili della pubblicazione delle revoke e dell'archiviazione dei certificati. Le funzioni di gestione includono la registrazione, l'inizializzazione (installazione dei materiali delle chiavi), il recupero della coppia di chiavi e la certificazione incrociata tra diverse CA.

Sicurezza Web e Minacce

Il Web richiede strumenti di sicurezza specifici a causa della complessità del software, del rischio che un server diventi un "trampolino di lancio" per attacchi interni e dell'inconsapevolezza degli utenti.

Il protocollo TLS (evoluzione di SSL) è progettato per fornire un servizio sicuro sopra il protocollo TCP. Si articola in due livelli:

1. **TLS Record Protocol:** Fornisce i servizi di base di confidenzialità e integrità.
2. **Protocolli di livello superiore:** Includono l'Handshake Protocol (per l'autenticazione e negoziazione delle chiavi), il Change Cipher Spec Protocol (per aggiornare la suite di cifratura) e l'Alert Protocol (per trasmettere avvisi e gestire errori "fatal").

Sessioni e Connessioni TLS

Viene fatta una distinzione fondamentale tra:

- **Sessione:** Un'associazione a lungo termine tra client e server creata tramite l'Handshake. Definisce i parametri crittografici (come il Master Secret di 48 byte) per evitare costose rinegoziazioni. **Una sessione potrebbe avere molteplici connessioni.**
- **Connessione:** Una relazione peer-to-peer transitoria associata a una sessione.

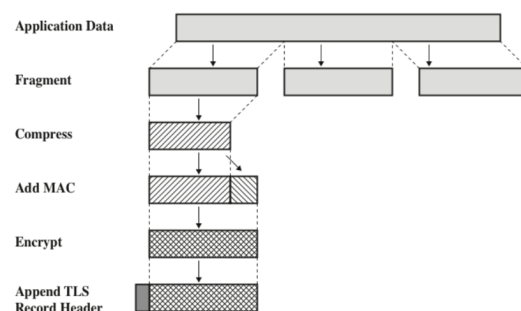
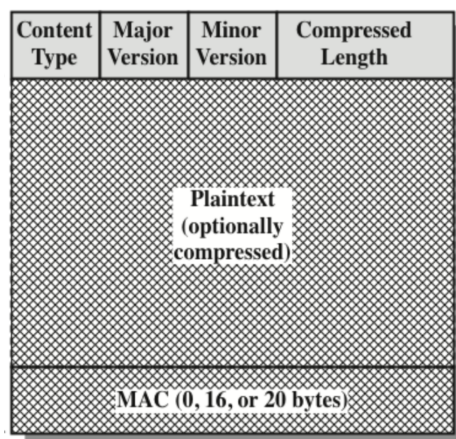


Figure 6.3 TLS Record Protocol Operation

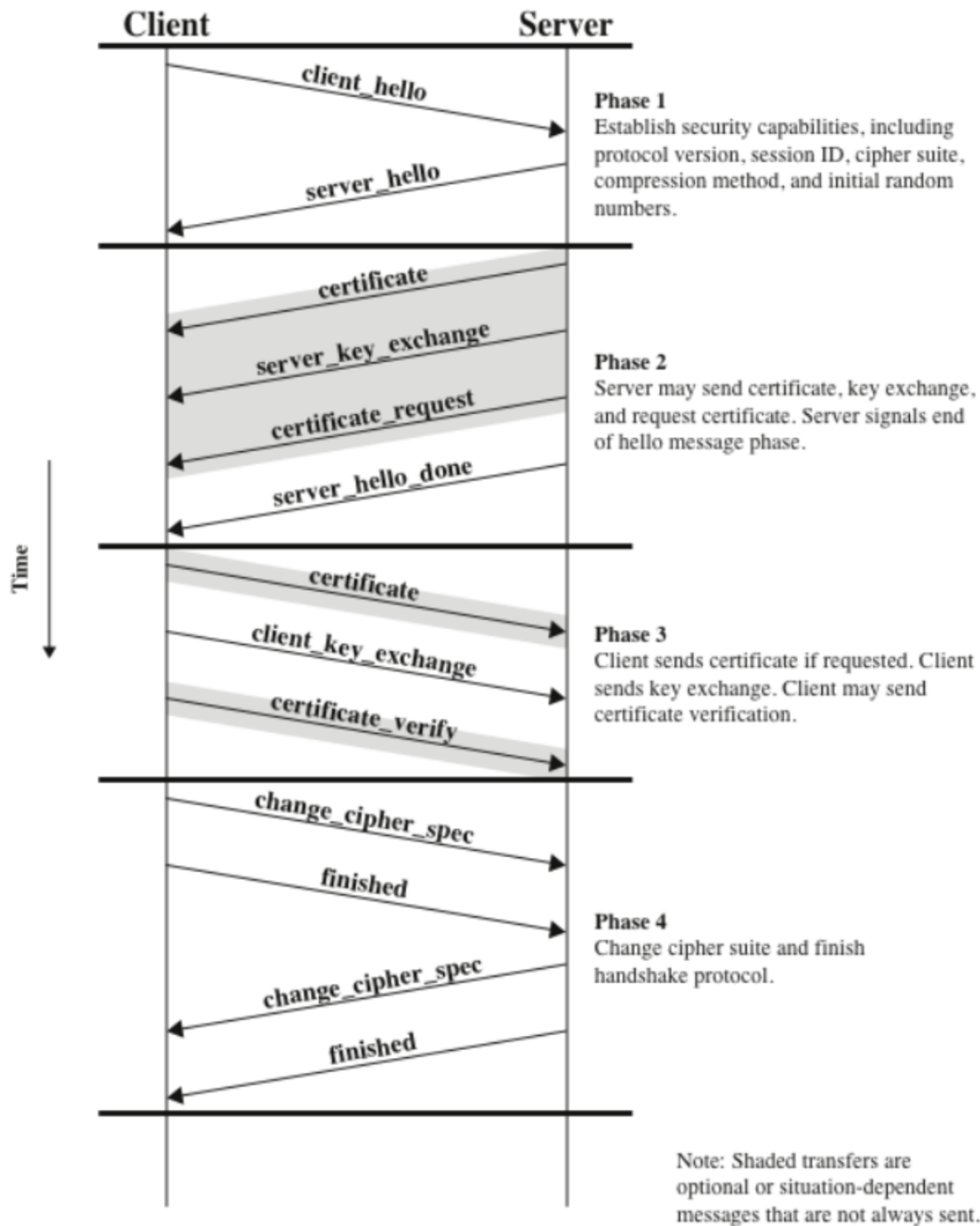
Funzionamento del TLS Record Protocol

Quando un messaggio applicativo deve essere trasmesso, il Record Protocol esegue i seguenti passaggi:



1. **Frammentazione:** I dati vengono divisi in blocchi di massimo 2^{14} byte.
2. **Compressione:** Operazione facoltativa che non deve far perdere dati.
3. **Calcolo del MAC:** Viene utilizzato l'algoritmo HMAC (con hash MD5 o SHA-1) per garantire l'integrità.
4. **Cifratura:** Il messaggio (compresso e con MAC) viene cifrato con algoritmi come AES o 3DES.
5. **Aggiunta dell'intestazione (Header):** Include il tipo di contenuto, la versione del protocollo e la lunghezza compressa. Questo avviene nel passaggio del livello successivo nella scala TCP/IP. Esempio: passando dal livello 4 al 5 (trasporto-applicazione) viene passato l'header del livello di trasporto a quello applicativo, che include ad esempio il protocollo usato (TCP/UDP).

handshake SSL/TLS tra client e server



Dettagli sul Livello di Trasporto (TCP/UDP):

- **Segmentazione:** La segmentazione dei dati avviene a questo livello perché il trasporto è il responsabile della gestione della comunicazione end-to-end tra le applicazioni.
- **Checksum UDP:** Per il controllo dell'integrità, l'UDP utilizza un campo di checksum a 16 bit (basato sull'aritmetica in complemento a 1). Tecnicamente, questo meccanismo:
 - Rileva tutti gli errori a 1 bit.
 - Rileva tutti gli errori a 3 bit (e in generale tutti gli errori dispari).
 - NON è in grado di rilevare tutti gli errori a 2 bit.
- **Differenza di affidabilità:** Mentre il TCP è un protocollo affidabile che non perde pacchetti (tramite handshake e riscontri), l'UDP privilegia la velocità e può perdere pacchetti, venendo scelto in base alle necessità dell'applicazione.

Oltre al record protocol il protocollo TLS include dei protocolli per gestire la comunicazione:

Change cipher Spec Protocol: mandando un byte passiamo dalla fase di handshake alla fase di trasferimento dei dati crittografati. L'unico scopo di questo messaggio è quello di far sì che lo stato in sospeso (pendente) venga copiato nello stato corrente, che aggiorna la suite di cifratura da utilizzare su questa connessione.

Alert protocol: segnala avvisi di emergenza con warning o fatal, nel caso fatal la connessione interrotta e la sessione non può più apprendere nuove connessioni.

Heartbeat Protocol: invio di segnali periodici per mantenere attiva la sessione.

TLS Handshake Protocol(4fasi)

Handshake per la negoziazione delle chiavi e autenticazione reciproca.

Fase1: (Security Capabilities) client manda un Hello inside della versione del TLS usata+ numeri casuali (nonce) per evitare replay attack), ID session, algoritmi di cifratura possibili. il server risponde con un hello impostando i parametri richiesti.

Fase 2: server invia il proprio certificato X.509 e può inviare un messaggio `server_key_exchange` per Diffie-Hellman o RSA conclude questa fase con `server_done`.

Fase 3 : il client verifica il certificato del server e se richiesto invia il proprio e un messaggio `client_key_exchange` con il segreto o i parametri Diffie-Hellman.

Fase 4: entrambi confermano la ricezione delle fasi precedenti e confermano che l'autenticazione si è riuscita.

Tipi di attacchi al SSL/TLS

Possiamo raggruppare questi attacchi in 4 categorie:

1. Attacchi al Handshake Protocol
2. 3. Attacchi al Record Protocol e agli Application Data Protocol
- Attacchi alla Public Key Infrastructure
4. Altri attacchi

HTTPS

HTTPS è HTTP che viaggia **dentro SSL/TLS**, quindi permette a browser e server di comunicare in modo sicuro. Per l'utente la differenza visibile è l'URL che inizia con `https://` e l'uso della **porta 443** invece della 80, ma il cambiamento reale è che tutto lo scambio HTTP viene protetto.

Con **HTTPS** vengono cifrati non solo i contenuti delle pagine, ma anche le **richieste**, i dati dei **form**, i **cookie** e persino gli **header HTTP**, impedendo intercettazioni e manipolazioni lungo il percorso.

Il funzionamento di base è questo: prima si stabilisce una normale **connessione TCP**, poi parte l'**handshake TLS** (inizia con il ClientHello) **per negoziare algoritmi e chiavi**; una volta completato l'handshake, le normali richieste e risposte HTTP vengono inviate all'interno del canale TLS, quindi protette.

Anche la chiusura ha regole precise. HTTP può indicare la fine con `Connection: close`, mentre TLS prevede una chiusura ordinata tramite lo scambio dell'avviso `close_notify` prima di chiudere TCP. Se la connessione viene interrotta senza questi passaggi, la chiusura è considerata anomala: può essere un semplice errore di rete, ma dal punto di vista della sicurezza può anche sembrare un comportamento sospetto, che il client dovrebbe almeno segnalare.

FTP

Il File Transfer Protocol (FTP) è un protocollo a **livello applicativo** utilizzato per il trasferimento di file tra host. La sua caratteristica tecnica distintiva è l'utilizzo di due canali separati (e quindi due porte diverse) per gestire la sessione:

- Porta 21 (Canale dei Comandi): Viene utilizzata per l'invio di istruzioni, come le credenziali di autenticazione (login) o le richieste di navigazione nelle cartelle.
- Porta 20 (Canale dei Dati): Viene utilizzata esclusivamente per il trasferimento fisico dei file.

Nota sulla sicurezza: FTP standard è considerato non sicuro, motivo per cui viene spesso sostituito da SFTP (che gira su SSH) per proteggere i dati e le credenziali.

SSH

Shh evoluzione di telnet e server per sessioni cifrate remote.

Caratteristiche dell'SSH: è una suite di tre protocolli che operano sopra il TCP

TLS: autenticazione del server, riservatezza e integrità dei dati usa coppie di chiavi per autenticare l'host garantisce la forward secrecy per rendere inattaccabili le sessioni passate se una chiave viene compromessa.

User Auth protocol: autentica l'utente sul server tramite password, chiave pubblica o altri metodi.

Connection protocol: i pacchetti ssh includono, lunghezza del pacchetto lunghezza del padding, payload, nonce, MAC (per l'integrità). Opera sopra il tunnel sicuro e permette il multiplexing quindi più canali su un'unica connessione. Ogni canale ha un numero unico e il flusso è controllato da un meccanismo a finestra. Vita del canale(tre fasi, apertura trasferimento e chiusura).

L'architettura di SSH si basa su tre protocolli principali che operano sopra il TCP

Transport layer protocol: che si occupa dell'autenticazione del server, la riservatezza e l'integrità dei dati.

Host keys : l'autenticazione si fa con una coppia di chiavi pubblica e privata., il client deve essere a conoscenza della pubblica del server a priori grazie a un database locale (trusted) oppure certificati rilasciati da un autorità.

Formato dei pacchetti: lunghezza, lunghezza del padding, payload, i padding causale e il MAC per l'integrità.

Fasi di scambio: prima fase scambio di stringhe con versione del protocollo e del software

Negoziazione degli algoritmi: sia client che server si accordano per l'algoritmo da utilizzare.

Scambio di chiavi: basato su Diffie_Hellman permette la condivisione di una chiave master e autentica il server con firma digitale.

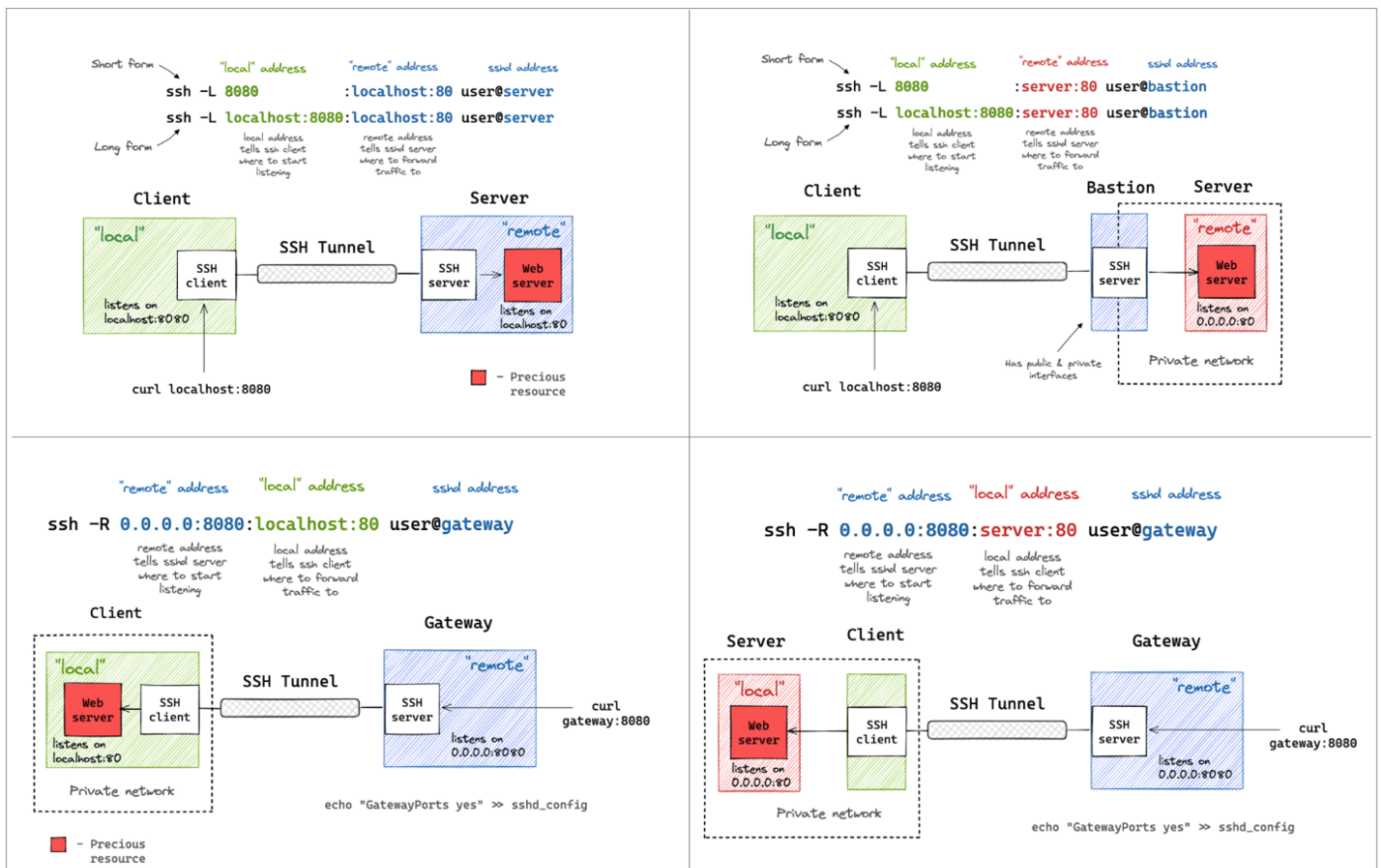
Richieste di servizio: il client richiede l'autenticazione utente o il protocollo di connessione.

PortForwarding: converte connessioni TCP non sicure in tunnel sicuri (tunneling)

Local: dirottamento del traffico di una porta locale verso una porta specifica del server remoto. Tutto i dati che vengono inviati vengono cifrati.

Remote: quando un firewall impedisce connessioni in entrata, tecnica che inoltra una porta su un server remoto verso una porta sulla tua macchina creando un tunnel sicuro per esporre servizi locali.

Esempio: local quando faccio ssh sul server oracle. Remote: quando espongo ristoflow attraverso Ngrok



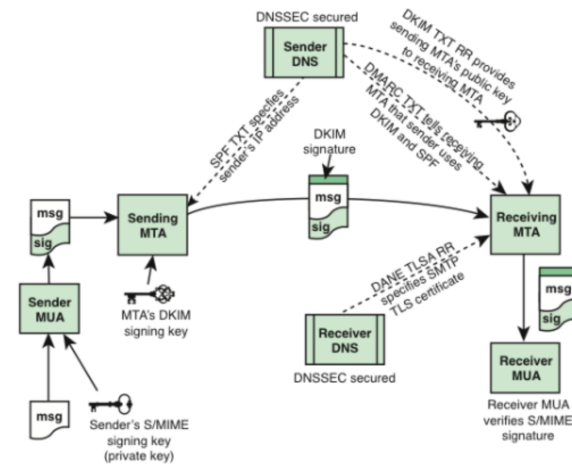
POSTA ELETTRONICA

Posta elettronica: le componenti di posta seguono lo standard RFC 5598

1. **MUA** (message user agent): corrisponde al client (programma di posta usato dall'utente.)
2. **MSA**(mail submission agent); riceve il messaggio dal MUA e applica le politiche del dominio.
3. **MTA** (message transfert agent). Il router delle email inoltra i messaggi ai vari server attraverso SMTP.
4. **MDA**(mail delivery agent) : consegna il messaggio dal sistema di trasferimento al magazzino messaggio.
5. **MHS**(message handling services) il mondo di trasferimento basato su MTA che spostano i messaggi sui server
6. **MS** (message store): dove le email vengono salvate accessibili via POP o IMAP.

Flusso tipico (a grandi linee):

MUA → server di invio (MSA) con SMTP (spesso 587, a volte 25) → MTA → MTA di destinazione (scelto via MX) → MDA
salva in mailbox → MUA recupera via IMAP/POP.



Protocolli di trasferimento:

SMTP (Simple Mail Transfer Protocol): basato su testo ASCII le transazioni avvengono tramite comandi HELO (identificazione) MAIL FROM (mittente) RCPT TO (destinatario), DATA (contenuto del messaggio da inoltrare). Con STARTTLS possiamo usare TLS per rendere sicuro SMTP.

POP3 vs IMAP3 : il primo per scaricare la posta usa la porta 110, IMAP gestisce messaggi e cartelle direttamente sul server porta 143 (es gmail). Nota (libero mail fornisce solo 2 gb di spazio quindi è consigliato usare POP 3 per scaricare la posta in locale.)

Formato dei messaggi mail RFC 5322: un messaggio è composto da header, riga vuota e body e message-ID.

TrustWorthy email (SP 800-177): cosa usare contro le minacce

- **STARTTLS**: mette SMTP sopra TLS (protezione del canale)
- **SMIME**: protegge contenuto mail firmando e/o cifrando.
- **DNSSEC**: protegge autenticità/integrità dei dati DNS
- **SPF**: che elenca gli indirizzi IP e i server autorizzati a inviare email per conto di quel dominio

Tutti i componenti sopra elencati garantiscono autenticità e integrità del messaggio.

MIME e S/MIME

Data la limitazione solo al testo ASCII a 7 bit è stato introdotto **MIME** (Multipurpose Internet Mail Extensions) per supportare più formati tra cui anche quelli multimediali e superare i limiti di dimensione. Introduce nuovi campi nell'intestazione come content-type e content-transfer-encoding. La codifica più usata è Base64. Utilizza codifiche di trasferimento per evitare alterazioni del contenuto.

S/MIME è la versione sicura di MIME garantisce la sicurezza attraverso 4 servizi principali:

Firma(RSA+SHA), compressione, cifratura(AES+ chiave scambiata con RSA), compatibilità email (consente la conversione nel formato desiderato).

Come funziona s/MIME: attraverso SHA e RSA:

1. Calcolo SHA del messaggio
2. Cifro con la chiave privata del mittente (firma)
3. Il destinatario decifra la firma con la chiave pubblica e li confronta se uguale OK

Come avviene la cifratura:

1. Genero una chiave simmetrica usa e getta
2. Cifro il messaggio con AES
3. Cifro la chiave simmetrica con la cova pubblica del destinatario
4. Il destinatario recupera la chiave con la sua chiave privata e decifra il messaggio

L'autenticità e la confidenzialità vengono garantite firmando il messaggio in chiaro poi cifrando sia messaggio che la firma. S/MIME consente entrambi gli ordini (prima firma poi cifrati o contrario). Il contrario potrebbe perdere alcune proprietà di legame tra firmatario e contenuto in chiaro.

Un e-mail può contenere non solo testo, molti sistemi accettano solo ASCII, S/MIME prevede una codifica a 7 bit per rendere trasportabile il contenuto.

S/MIME ha un campo per specificare il tipo di contenuto (content-type) che serve per trasportare dati firmati e/o cifrati.

Un messaggio S/MIME può contenere signedData (firma digitale: autenticità + integrità)

EnvelopedData (cifratura: confidenzialità). In caso di cifratura con la flag RecipientInfo possiamo recuperare le informazioni necessarie per trovare la chiave e decifrare.

Per recuperare un **messaggio firmato** il destinatario verifica la firma con il certificato del mittente, per un **messaggio cifrato** il destinatario usa la propria chiave cifrata per decifrare la chiave di sessione e poi il contenuto.

Ogni coppia di chiavi deve essere supportata da un certificato (X.509) firmato da una CA, l'utente mantiene un elenco locale di certificati attendibili.

Un protocollo di sicurezza e-mail alternativo è Pretty Good Privacy (PGP), che ha essenzialmente le stesse funzionalità di S/MIME. Nel PGP a differenza di S/MIME non abbiamo la chiave pubblica del mittente e quindi deve ottenerla in altri modi (pubblicate su siti Web protetti da TLS).

SP 800-177 raccomanda l'uso di S/MIME piuttosto che PGP a causa della maggiore fiducia nel sistema CA di verifica delle chiavi pubbliche.

DNS: database distribuito contenente name-server e resolver, i record tipici sono A/AAAA, MX, CNAME. MX è utilizzato per la posta (l'MTA usa il record MX del dominio per sapere a quale mail server consegnare).

Come funziona: quando cerchiamo una pagina web online la cerchiamo attraverso il suo nome. Per convertire il nome in indirizzo IP utilizziamo il DNS. Abbiamo quindi il resolver che chiede al DNS locale, se non è nella cache del locale interroga gli altri server fino ad arrivare alla root. Salva l'IP desiderato in cache con il relativo time to live, se non lo trova dà errore.

Approfondimento: La Gerarchia Radice (Root Nameservers)

Al vertice della struttura gerarchica del DNS operano i Root Nameserver, che rappresentano il punto di partenza per ogni risoluzione di nome non presente in cache.

- I 13 Root Server: Esistono 13 nomi logici di root server, identificati dalle lettere dalla A alla M (nella forma lettera.root-servers.net). Sebbene siano solo 13 nomi, grazie alla tecnologia Anycast le interrogazioni vengono distribuite su centinaia di server fisici situati in tutto il mondo.

- Funzione: I root server gestiscono il punto più alto della gerarchia e non contengono gli indirizzi IP dei singoli siti, ma indicano al resolver quali sono i server autorevoli per i Top-Level Domain (TLD) (es. .it, .com, .net, .org, .gov). Ad esempio, per il dominio .it, il root server risponderà indicando di consultare i server della gerarchia italiana.

- Operatori e Luoghi: Questi server sono gestiti da diverse organizzazioni internazionali. Tra gli operatori più rilevanti citati a lezione figurano:

- VeriSign: gestisce i server A (storicamente il primo) e J.

- NASA: gestisce il server E.

- ICANN: gestisce il server L.

- Altri operatori includono università (Maryland, USC), enti militari (U.S. Army, DISA) e operatori di rete (Cogent, WIDE).

Il DNS classico è vulnerabile ad attacchi del tipo **man-in-the-middle** oppure **DNS-poisoning**. Per questo utilizziamo DNSSEC che aggiunge firme digitali ai record per garantire origine e integrità delle risposte.

RFC 4034 definisce 4 nuovi record di risorse DNS (DNSSEC):

DNSKEY: record che contiene la chiave di firma pubblica

RRSIG: record che contiene la firma crittografica per un set di record associato

NSEC: record autenticato di negazione dell'esistenza

DS: contiene l'hash di DNSKEY

RRSET

Il primo passo nella configurazione di un sistema DNSSEC è raggruppare tutti i record di risorse (RR) in un RRset. L'RRset raggruppa tutti i record dello stesso tipo ed etichetta in un

unico bundle che può essere successivamente firmato. Questo raggruppamento riduce notevolmente il traffico necessario per verificare e considerare attendibile ciascun record, poiché i dati crittografici devono essere estratti solo una volta per tutti i record dello stesso tipo.

DNSSEC: ruoli di ZSK e KSK e catena di fiducia

In DNSSEC ogni zona ha una **coppia** di chiavi **ZSK**: la **ZSK privata** firma i record DNS della zona (i gruppi di record si chiamano RRset) e **la firma viene salvata come record RRSIG**.

Quando un resolver chiede, ad esempio, un record A/AAAA, il server risponde con RRset + RRSIG, così il resolver può controllare che la **risposta non sia stata modificata**.

Per fare questo controllo il resolver deve conoscere la **ZSK pubblica, che il server pubblica in un record DNSKEY**.

Ma fidarsi “a occhi chiusi” della ZSK pubblica è rischioso: per questo esiste la **KSK**, che **firma il RRset DNSKEY e quindi “certifica” la ZSK**.

Il resolver verifica prima i record richiesti con la ZSK pubblica, poi verifica che la ZSK sia autentica controllando la firma del DNSKEY con la KSK pubblica.

Resta un problema: **la KSK da sola non basta**, perché se arriva sempre dallo stesso server potrebbe essere stata manomessa.

Qui entra il **record DS**: la zona figlia **calcola l'hash della propria KSK pubblica e lo fa pubblicare nella zona genitore come DS**.

Quando il resolver scende nella zona figlia, prende il DS dal genitore e lo confronta con l'hash della KSK: **se combacia, la KSK è considerata valida**.

A quel punto il resolver può fidarsi della KSK, quindi della ZSK, quindi di tutti i record della zona figlia: **questa è la catena di fiducia**.

Infine **anche il DS è firmato nel genitore** (ha il suo RRSIG), **così la fiducia può risalire gerarchicamente fino alla radice**.

DANE e TLSA

DANE: protocollo di sicurezza internet che consente ai certificati digitali X.509 di essere associati ai nomi di dominio utilizzando DNSSEC. E' visto come un modo per autenticare le entità client/server TLS senza una CA. Avere troppe CA può portare problemi di sicurezza. In passato sono stati emessi certificati per domini noti a coloro che non possedevano tali domini.

DANE consente di certificare le chiavi utilizzate nei client o nei server TLS di quel dominio, memorizzandole nel DNS. Ha bisogno che i record DNS siano firmati con DNSSEC.

Consente di specificare quale CA è autorizzata ad emettere certificati per una determinata risorsa.

Definisce un nuovo tipo di record DNS, il **TLSA** per autenticare in maniera sicura certificati SSL/TLS.

L'idea di DANE è semplice: “Il dominio, dentro DNS firmato (DNSSEC), dichiara quale certificato o quale chiave è valida per quel servizio TLS”. Per farlo introduce il record TLSA, che lega un servizio (es. smtp.example.com:25 o example.com:443) a:

- un certificato specifico,
- oppure una CA specifica,
- oppure direttamente una public key.

DANE può essere usato insieme a SMTP su TLS per garantire una consegna della posta elettronica più sicura.

Come si utilizza DNSSEC su S/MIME: viene utilizzato per proteggere in modo più completo la consegna della posta elettronica.

SPF indica, tramite un record TXT nel DNS, quali indirizzi IP o server sono autorizzati a spedire email per un determinato dominio (dominio del MAIL FROM / Return-Path).

Nel record SPF possono comparire meccanismi diretti come **ip4** e **ip6**, oppure meccanismi che richiedono ulteriori lookup come **mx** (dice che anche i server elencati come MX del dominio sono mittenti validi) e **include** (“fidati anche delle regole SPF di un altro dominio”). Questo è comunissimo quando usi servizi terzi come Google Workspace, Microsoft 365, Mailchimp, CRM, ecc.). Usando dig -t TXT dominio è possibile visualizzare la policy SPF

pubblicata. Il controllo SPF viene effettuato dal server ricevente confrontando l'IP del mittente con la policy del dominio del MAIL FROM.

SPF lato ricevente (come viene controllato):

Quando un server ricevente supporta SPF, fa così:

1. guarda il dominio della busta SMTP, cioè quello nel comando MAIL FROM: (attenzione: non è per forza lo stesso del campo "From:" visibile all'utente)
2. prende l'IP del server che gli sta consegnando la mail
3. interroga il DNS per il record TXT SPF del dominio trovato
4. applica in ordine i meccanismi finché trova un match e produce un esito (pass/fail/softfail/neutral...)

Quindi controlla se il dominio specificato nell'SMTP è accettato o meno dalla nostra casella di posta (attraverso la regola per la casella di posta all'interno del server DNS).

DKIM: applica una firma digitale al messaggio per garantirne l'integrità e associare la mail a un dominio.

Il server mittente calcola un hash di header e body selezionati e firma tale hash con la chiave privata del dominio. La chiave pubblica è pubblicata nel DNS come record TXT (selector._domainkey). Il server ricevente recupera la chiave pubblica dal DNS e verifica che la firma sia valida.

DMARC: è un pezzo diverso rispetto a DNSSEC/DANE: non riguarda "proteggere il DNS" o "autenticare TLS", ma riguarda il problema reale più comune della posta: **spoofing** del mittente (email che sembrano provenire da un dominio ma non lo sono).

Prima di DMARC esistevano già:

- SPF, che controlla se l'IP che sta inviando la mail è autorizzato dal dominio (guardando un record DNS SPF).
- DKIM, che firma il messaggio e lega firma/integrità a un dominio firmatario.

DMARC definisce una policy che indica come gestire i messaggi quando SPF e/o DKIM non producono un risultato valido e allineato con il dominio del campo From visibile.

Perché DMARC passi, è necessario che:

- **SPF passi** e il dominio del MAIL FROM sia **allineato con il dominio nel campo From**, oppure
- **DKIM passi** e il dominio d= della firma sia **allineato con il campo From**.

Se nessuna di queste condizioni è soddisfatta, DMARC fallisce e applica la policy configurata (none, quarantine, reject). DMARC consente inoltre l'invio di report aggregati sull'autenticazione dei messaggi.

La Decisione di Instradamento (Routing Decision): perché una macchina possa comunicare correttamente in rete, deve essere configurata con quattro parametri fondamentali: **indirizzo IP**, **subnet mask**, **default gateway** e **server DNS**. La prima operazione logica eseguita dall'host per inviare un messaggio è capire se la destinazione si trova nella propria rete locale o in una rete esterna.

• **Analisi Binaria:** Il sistema converte in binario sia il proprio indirizzo IP che l'indirizzo IP di destinazione.

• **Applicazione della Subnet Mask:** Viene applicata la maschera di rete (es. /24, che indica che i primi 24 bit identificano la rete) per confrontare le porzioni di rete dei due indirizzi.

- Se i bit di rete coincidono, la destinazione è locale.
- Se i bit non coincidono, la destinazione è esterna.

Movimento Fisico dei Dati: Il Protocollo ARP (Livello 2) Una volta stabilito dove si trova il destinatario, la comunicazione passa dal livello di rete (IP) al livello Data Link (Livello 2), dove i dati si muovono fisicamente tra schede di rete (hardware) tramite frame. Ogni scheda di rete è identificata in modo univoco da un MAC Address, composto da 6 coppie di numeri esadecimali.

• **Destinazione Locale (Broadcast ARP):** Se la macchina è nella stessa sottorete, il mittente invia un pacchetto di broadcast ARP ("chi ha questo indirizzo IP?") a tutti i dispositivi della LAN. La macchina che possiede quell'IP risponde fornendo il proprio MAC Address, che il mittente salva in cache per completare l'invio fisico.

• **Destinazione Esterna (Default Gateway):** Se la destinazione è fuori dalla sottorete, l'host invia il pacchetto al Default Gateway (solitamente l'indirizzo IP del router di uscita), che si occuperà di instradarlo verso le reti esterne.

Vulnerabilità del Protocollo ARP: questa logica è vulnerabile ad attacchi di sicurezza: poiché il protocollo ARP non prevede autenticazione, un attaccante potrebbe inviare risposte ARP false spacciandosi per un altro dispositivo (es. il server DNS o il gateway). In questo caso, al broadcast ARP potrebbero rispondere due macchine diverse, permettendo all'attaccante di intercettare o deviare il traffico (ARP Spoofing).

Note Tecniche sui Dispositivi:

- I Router operano principalmente al livello 3 (IP) per gestire l'instradamento, ma si interfacciano con il livello 4 per il trasporto.
- Gli Switch operano al livello 2 (Data Link), gestendo il traffico basandosi sui MAC Address delle schede di rete.

Sicurezza a livello IP e IPsec: architettura, funzionalità e applicazioni

IPsec

La sicurezza può essere implementata a livello applicativo ma non basta, è necessario adottare tecniche di sicurezza anche nei livelli più bassi.

Per questo si adotta la sicurezza a livello IP, che consente di proteggere tutto l'intero traffico di rete, indipendentemente dall'applicazione usata

La sicurezza IP si basa su tre funzioni principali:

Autenticazione: che garantisce l'identità della sorgente e l'integrità dei pacchetti.

Confidenzialità: che protegge i dati tramite crittografia

Gestione delle chiavi: necessaria per lo scambio sicuro delle chiavi crittografiche.

Questi principi sono stati formalizzati nel 1994 dall'IAB(RFC 1636) e hanno portato allo sviluppo dell'IPsec. IPsec permette di realizzare VPN sicure, accesso remoto protetto, comunicazioni sicure tra organizzatori ed un rafforzamento della sicurezza del commercio elettronico.

Caratteristica principale IPsec

È la capacità di crittografare e/o autenticare tutto il traffico a livello IP, solitamente il traffico Ip non protetto circola all'interno delle LAN, mentre quello che attraversa le WAN è protetto tramite IPsec operante su dispositivi come router o firewall. IPsec è ampiamente usato anche per l'accesso remoto sicuro. Dipendenti in viaggio o in telelavoro possono collegarsi alla rete aziendale tramite Internet utilizzando IPsec, autenticandosi e cifrando tutto il traffico scambiato. Questo consente di lavorare in sicurezza da qualunque posizione, riducendo i costi di connessione e mantenendo lo stesso livello di protezione disponibile in sede.

Benefici

Controllo accessi: lascia passare solo il traffico "permesso", implementato in un firewall

Autenticazione dell'origine: verifica chi ha mandato il pacchetto.

Anti-replay blocca pacchetti vecchi , ripetuti da un attaccante.

Confidenzialità: cifra i dati

Riservatezza del del flusso di traffico: cerca di rendere più difficile capire quanto traffico sta transitando e quando.

IPsec e routing

IPsec è rilevante per il routing perché protegge il piano di controllo, cioè i messaggi che i router si scambiano per costruire e aggiornare le tabelle di instradamento. Senza protezione, un attaccante potrebbe inviare annunci di routing falsi (facendo deviare il traffico), modificare aggiornamenti legittimi o generare ICMP Redirect malevoli.

Con IPsec puoi:

- autenticare i messaggi (sai che arrivano davvero da un router autorizzato);
- garantire integrità (non sono stati alterati);
- opzionalmente avere riservatezza (se cifri anche il contenuto).

Nel caso di OSPF, l'idea è che i pacchetti OSPF (Hello, LSU, LSA, ecc.) vengano accettati solo se associati a una Security Association (SA) valida: in pratica, OSPF "si fida" di chi ha le chiavi/parametri IPsec corretti.

IPsec è definito da un insieme complesso di RFC I **Documenti IPsec** possono essere classificati nei seguenti gruppi:

1. **Architettura**
2. **AH**(Authentication Header) deprecato adesso si usa **ESP**
3. **ESP** è costituito da un'intestazione e un trailer di incapsulamento utilizzati per fornire crittografia o crittografia/autenticazione combinata.
4. **IKE(internet Key Exchange)**: si tratta di una raccolta di documenti che descrivono gli schemi di gestione delle chiavi da utilizzare con IPsec.
5. **Algoritmi crittografici**: documenti che definiscono e descrivono algoritmi crittografici che vengono usati per la crittografia, per l'autenticazione dei messaggi e lo scambio di chiavi.

I due protocolli principali:

IPsec fornisce sicurezza a livello IP permettendo di scegliere servizi, algoritmi e chiavi per proteggere i pacchetti. Per farlo utilizza due protocolli principali.

AH (Authentication Header) serve a garantire autenticità dell'origine e integrità dei dati, assicurando che il pacchetto provenga da un mittente legittimo e non sia stato modificato; include anche protezione contro i replay, ma non cifra il contenuto.

ESP (Encapsulating Security Payload) fornisce confidenzialità tramite cifratura del payload e può anche offrire autenticazione, integrità e protezione anti-replay. È il protocollo IPsec più usato perché combina cifratura e sicurezza dei dati.

Due modalità

Transport mode: protegge solo i dati dentro il pacchetto IP(il contenuto) non tutto. Si usa solitamente per la comunicazione end-to-end tra due host (tipo client-server).

ESP in transport mode cifra il contenuto ma non l'header;

AH: autentica il contenuto e alcune parti selezionate dell'header.

Tunnel mode: protegge l'intero pacchetto IP.

Il pacchetto viene messo all'interno di un nuovo pacchetto con un nuovo header IP esterno (incapsulazione). Si usa solitamente tra due Gateway per collegare due reti in modo sicuro (VPN site-to-site). I router nel mezzo vedono solo l'header esterno, non possono leggere il contenuto interno.

ESP cifra tutto il pacchetto interno, compreso il suo header IP.

AH autentica tutto il pacchetto e alcune parti dell'header esterno.

IPsec per ogni pacchetto decide se lasciarlo passare in chiaro, bloccarlo, proteggerlo con AH/ESP e con quali parametri. Questa decisione nasce dall'uso combinato di due database:

SPD (Security Policy DB) dice che cosa bisogna fare a quel tipo di traffico.

SAD (Security Association DB) contiene i dettagli tecnici per eseguire davvero ESP/ AH (chiavi, algoritmi, contatori).

Un concetto importante relativo all'autenticazione e confidenzialità per l'IP è:

SA(security association) è una relazione logica unidirezionale tra un mittente e un destinatario che definisce come proteggere un flusso di traffico. Per una protezione bidirezionale servono due SA.

Un SA è identificato da 3 elementi:

SPI (security parameters index) numero a 32 bit, etichetta locale per riconoscere le SA. Viene trasportato nelle intestazioni AH e ESP per il controllo.

Indirizzo IP di destinazione: endpoint finale della SA

Protocollo di sicurezza: indica se la SA usa AH o ESP

SAD : contiene una voce per ogni SA con tutti i parametri operativi necessari per elaborare i pacchetti (Es: SPI per associare il pacchetto alla SA giusta).

Un'associazione di sicurezza è definita dai seguenti parametri:

1. **Sequence Number Counter**: contatore usato per numerare i pacchetti (anti-raplay)
2. **Gestione overflow del contatore**: se il contatore raggiunge il limite definisce se impedire ulteriore trasmissione di pacchetti sulla SA.

3. **Antireplay window:** finestra per capire se un pacchetto in arrivo è un replay (duplicato).
4. **AH Information:** algoritmo di autenticazione, chiavi, durate delle chiavi e parametri correlati utilizzati con AH (richiesti per le implementazioni AH).
5. **ESP Information:** algoritmo di crittografia e autenticazione, chiavi, valori di inizializzazione, durata delle chiavi e parametri correlati utilizzati con ESP (richiesti per le implementazioni ESP).
6. **Lifetime della SA:** dopo un certo tempo o dopo tot di byte, la SA va sostituita o terminata.
7. **Modalità IPsec:** transport/tunnel
8. **Path MTU:** dimensione massima del pacchetto senza frammentazione.

SPD: è un insieme di regole che classificano il traffico tramite selettori (campi usati come filtri) ogni regola SPD descrive un sottoinsieme di traffico e l'azione:

Protect: usa IP sec

ByPass: non usare IPsec

Discard: scarta

Come funziona l'elaborazione in uscita:

Per ogni pacchetto in uscita:

1. Match dei valori del pacchetto con quelli dell'SPD (trovo la regola che si applica)
2. Determinare l'eventuale SA e quindi la relativa SPI usare
3. Applico l'elaborazione IPsec richiesta.

Selettori tipici che determinano una voce SPD:

IP remoto (destinazione),

IP locale (sorgente),

Next layer Protocol: quale protocollo sopra IP usare (tcp/udp ecc..),

Nome utente: non è nel pacchetto, ma può essere noto all'host.

Porte locali/remota: TCP/UDP

Come funziona l'elaborazione del traffico IP?

Pacchetti in uscita

Un blocco di dati da uno layer superiore, come TCP, viene passato allo strato IP e viene formato un pacchetto IP, costituito da un'intestazione IP e da un body IP. Quindi si verificano i seguenti passaggi:

1. **IPsec cerca nell'SPD una corrispondenza con questo pacchetto.**
2. **Se non viene trovata alcuna corrispondenza**, il pacchetto viene **scartato** e viene generato un messaggio di errore.
3. Se viene **trovata una corrispondenza**, l'ulteriore elaborazione è determinata dalla prima voce corrispondente nell'SPD. Se la politica per questo pacchetto è **DISCARD**, il pacchetto verrà scartato. Se la policy è **BYPASS**, non vi è alcuna ulteriore elaborazione IPsec; il pacchetto viene inoltrato alla rete per la trasmissione. Se la policy è **PROTECT**, viene effettuata una ricerca nel SAD per una voce corrispondente. Se non viene trovata alcuna voce, viene richiamato IKE per creare una SA con le chiavi appropriate e viene creata una voce nella SA.
4. La voce corrispondente nel SAD determina l'elaborazione di questo pacchetto. È possibile eseguire la crittografia, l'autenticazione o entrambe e utilizzare la modalità trasporto o tunnel. Il pacchetto viene quindi inoltrato alla rete per la trasmissione.

Pacchetti in entrata

Un pacchetto IP in entrata attiva l'elaborazione IPsec. Si verificano i seguenti passaggi:

1. **IPsec determina se si tratta di un pacchetto IP non protetto** o di un pacchetto con **intestazioni/trailer ESP o AH**, esaminando il campo Protocollo IP (IPv4) o il campo

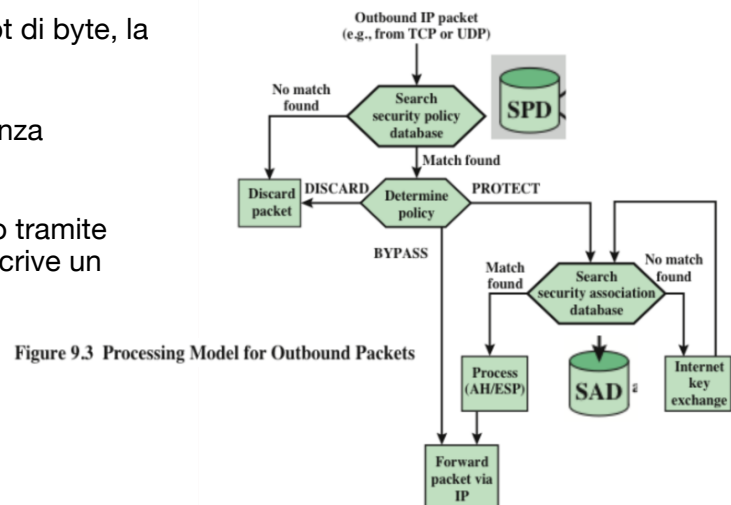


Figure 9.3 Processing Model for Outbound Packets

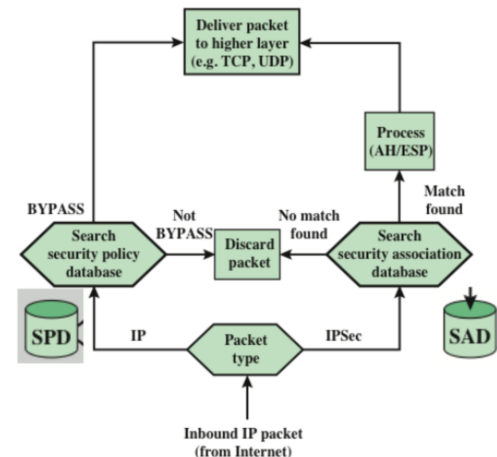


Figure 9.4 Processing Model for Inbound Packets

Intestazione successiva (IPv6).

2. Se il **pacchetto non è protetto**, **IPsec cerca nell'SPD una corrispondenza con questo pacchetto**. Se la prima voce corrispondente ha una politica di BYPASS, l'intestazione IP viene elaborata e rimossa e il corpo del pacchetto viene consegnato al livello successivo più alto, come TCP. Se la prima voce corrispondente ha una politica di PROTECT o DISCARD, o se non esiste alcuna voce corrispondente, il pacchetto viene scartato.

3. Per un pacchetto **protetto**, **IPsec cerca nel SAD**. Se non viene trovata alcuna corrispondenza, il pacchetto viene scartato. In caso contrario, IPsec applica l'elaborazione ESP o AH appropriata. Quindi, **l'intestazione IP viene elaborata e rimossa e il corpo del pacchetto viene consegnato al livello successivo superiore**, come TCP.

ESP può dare:

Confidenzialità (cifratura)

Autenticazione + integrità

Anti-replay (contro pacchetti "ricopiati e reinviati")

Riservatezza del flusso (limitata: rende più difficile capire la reale dimensione/andamento del traffico).

Formato di un pacchetto ESP (campi essenziali)

Un pacchetto ESP ha:

SPI (32 bit): identifica la SA.

Sequence Number (32 bit): contatore crescente per anti-replay.

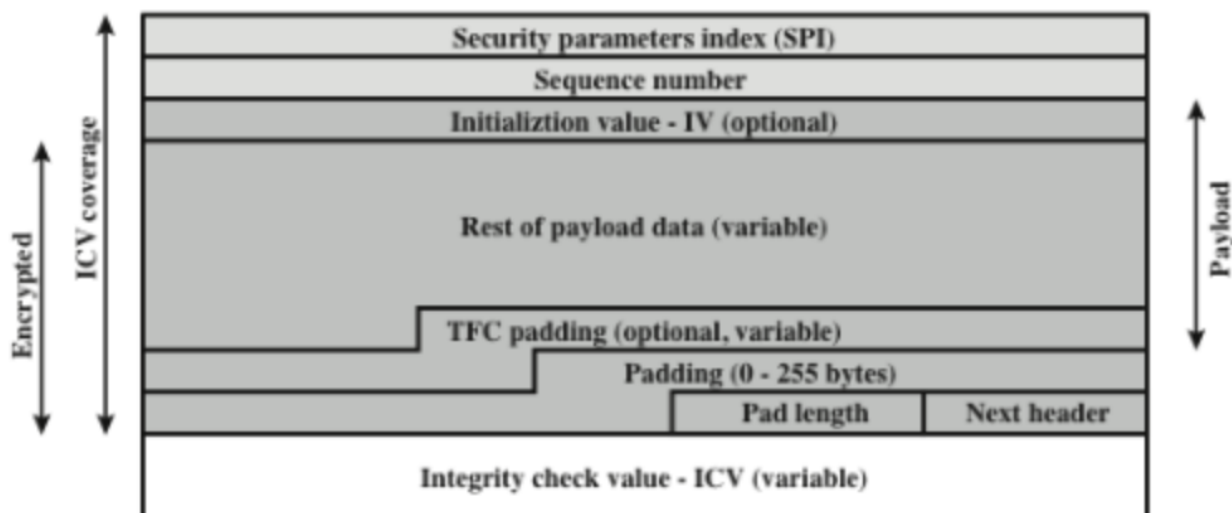
Payload Data (0-255 byte): ciò che viene protetto

Padding: byte aggiunti.

Pad Length: (8 bit) quanti byte di padding.

Next Header: identifica il tipo di dati contenuti nel campo dati del payload identificando la prima intestazione in quel payload (ad esempio, un'intestazione di estensione in IPv6 o un protocollo di livello superiore come TCP).

ICV (Integrity Check Value): valore di integrità/autenticazione (facoltativo, dipende dall'algoritmo/SA).



(b) Substructure of payload data

Nel payload possono essere presenti due campi aggiuntivi. Un initialization value (IV), o nonce, è presente se richiesto dalla crittografia o dall'algoritmo di crittografia autenticato utilizzato per ESP.

In ESP i campi **Payload Data**, **Padding**, **Pad Length** e **Next Header** vengono cifrati.

Se l'algoritmo di cifratura lo richiede, un **IV può essere inserito all'inizio del payload** e in genere **non è cifrato**.

Il campo ICV è opzionale ed è presente solo se è abilitato il servizio di integrità.

L'**ICV viene calcolato dopo la cifratura**, permettendo al destinatario di rilevare e scartare rapidamente pacchetti falsi o riprodotti, riducendo l'impatto di attacchi DoS.

Poiché l'**ICV non è cifrato**, deve essere **calcolato usando un algoritmo di integrità con chiave**.

Il padding è necessario perché alcune cifrature richiedono una lunghezza multipla di un certo blocco, ESP vuole certi campi allineati (multipli 32 bit), padding extra per rendere meno evidente la vera dimensione dei dati.

Attacco anti replay

Un attaccante copia un pacchetto valido e lo rinvia. Il mittente usa un contatore che parte da 0 e per il primo pacchetto usa 1 ecc.. se l'anti replay è attivo non si può ricominciare da zero, si arriva ad un massimo di $2^{32} - 1$, quindi raggiunto il massimo la SA va chiusa e se ne negozia una nuova. Il destinatario usa una finestra scorrevole, e tiene traccia del numero più alto valido ricevuto(N), scarta duplicati o troppo vecchi, infine se arriva un pacchetto con un numero più alto N, sposta la finestra in avanti

Una SA può usare **AH** o **ESP** ma non tutti e due insieme nella stessa SA.

Quindi se per lo stesso traffico vuoi : confidenzialità + autenticazione/integrità oppure vuoi protezioni diverse su tratte diverse devi usare più SA sullo stesso flusso, questa catena prende il nome di **Security Association Bundle**: che sarebbe una **sequenza di elaborazioni IPsec** che un pacchetto deve subire per ottenere i servizi desiderati.

Abbiamo due modi per impacchettare più SA

- **Transport adjacency**: applichi più protocolli di sicurezza allo stesso pacchetto IP, senza fare tunnelling esempio ESP in transport + AH in transport sopra. Limite dovuto al fatto che nidificare non dà vantaggi extra .
- **Iterated Tunneling**: usi più tunnel IP uno dentro l'altro. Ogni livello di tunnel può avere endpoint diversi (Es più Gateway lungo il percorso). In questa modalità puoi avere più livelli di annidamento con senso operativo.

NOTA BENE:

AH ed ESP.

- **AH** serve a garantire autenticazione e integrità dei pacchetti (cioè che arrivino davvero dal mittente e non siano stati modificati), ma non fornisce confidenzialità: non cifra.
- **ESP** invece nasce soprattutto per la cifratura del contenuto, e può anche aggiungere integrità e autenticazione; quindi, usando ESP con autenticazione, in pratica puoi ottenere sia confidenzialità sia protezione contro manomissioni con un unico protocollo.

Quanto alle modalità: in **transport mode** viene protetto principalmente il payload del pacchetto, mentre l'header IP resta in chiaro (utile tipicamente end-to-end tra host). In **tunnel mode** invece si incapsula e protegge l'intero pacchetto IP originale, che diventa il payload di un nuovo pacchetto: così l'interno è cifrato (header incluso) ed è la modalità tipica per VPN tra gateway.

Autenticazione sul cyphertext: prima cifri, poi fai il controllo di integrità sui dati cifrati. Quindi il destinatario può scartare pacchetti falsi senza fidarsi del contenuto.

come concatenare più Security Association (SA)

Nel caso ESP + AH in transport (ESP "interno", AH "esterno") applichi due SA sullo stesso pacchetto in modalità transport. Prima **ESP cifra il payload** (senza autenticazione), poi **AH aggiunge autenticazione e integrità sopra**, proteggendo il payload cifrato (quindi anche ESP) e anche le parti non mutabili dell'header IP. Il vantaggio è che, **oltre alla confidenzialità** del payload, **ottiene autenticazione "forte"** che copre anche informazioni dell'header; lo **svantaggio** è più **overhead** e soprattutto **più complessità operativa**, perché devi negoziare e mantenere due SA.

Nel caso AH poi ESP in tunnel (AH “interno”, ESP “esterno”) prima applichi AH (tipicamente in transport sul pacchetto originale), così garantisca integrità e autenticazione del contenuto e di parti dell’header IP originale; poi applichi ESP in tunnel, che incapsula e cifra l’intero pacchetto già autenticato, aggiungendo un nuovo header IP esterno. Il risultato è che l’autenticazione (AH) resta dentro il tunnel cifrato, quindi non è facilmente osservabile o manipolabile dall’esterno, e in più nascondi completamente header e payload originali. È una scelta tipica quando vuoi una VPN “pulita”: ciò che viaggia fuori è solo il nuovo pacchetto esterno, mentre tutto il resto è protetto dentro il tunnel.

Quattro casi base di architettura IPsec

Nel caso **host ↔ host** (end-to-end) i due endpoint finali fanno direttamente IPsec: è la protezione più “pura” perché vale dalla sorgente alla destinazione, e puoi usare AH/ESP (tipicamente in transport, ma anche in tunnel se vuoi incapsulare).

Nel caso **gateway ↔ gateway** (site-to-site) gli host interni non fanno nulla: la cifratura e l’autenticazione avvengono tra i due firewall/router di confine, quasi sempre con tunnel mode, così si protegge “in blocco” il traffico tra due reti.

Nel caso **gateway ↔ gateway** più **host ↔ host**, mantieni il tunnel tra i gateway per la protezione generale, ma per alcune comunicazioni aggiungi un ulteriore livello end-to-end tra host, ottenendo una protezione stratificata (host-to-host dentro il tunnel).

Nel caso **host remoto ↔ gateway** aziendale (remote access VPN) il peer esterno è un singolo utente: si crea un tunnel tra il client remoto e il firewall aziendale per far entrare l’utente nella rete interna in modo sicuro.

Tutto questo però richiede **Security Association (SA)** e quindi chiavi e parametri coerenti. La gestione manuale delle chiavi è possibile solo in scenari piccoli e statici; appena crescono peer, scadenze e rotazioni, diventa impraticabile. Qui entra **IKE** (Porta 500)/**IKEv2**, che negozia automaticamente algoritmi, chiavi e crea le SA quando servono. **IKE usa Diffie–Hellman per derivare un segreto condiviso senza scambiarlo prima**, ma DH da solo non autentica le parti (rischio MITM) ed è costoso (rischio DoS). Per questo **IKE aggiunge difese come cookie anti-DoS, nonce anti-replay** e soprattutto autenticazione dello scambio.

Storicamente, lo “stack” di riferimento è stato **ISAKMP/Oakley**: ISAKMP fornisce il framework e i formati/messaggi per negoziare attributi di sicurezza e gestire SA e chiavi; **Oakley** è il **meccanismo di determinazione delle chiavi basato su Diffie–Hellman**, arricchito con **misure di sicurezza e flessibile nei formati**. In pratica: **ISAKMP** è l’**impalcatura di negoziazione e gestione**, **Oakley** è il “motore” di key exchange richiesto nelle prime specifiche.

Con IKEv2 quei nomi (ISAKMP/Oakley) non si usano più, e ci sono differenze importanti rispetto a IKEv1, ma la logica di base resta la stessa: **IKEv2 stabilisce prima una IKE SA** (un canale di **controllo protetto dove si negoziano parametri e si fa Diffie–Hellman**) e poi, dentro quel canale, **autentica le parti e crea una o più Child SA**, cioè le SA “di traffico” usate davvero da ESP/AH per proteggere i pacchetti.

INTRUSIONI NEI SISTEMI IN RETE E DI SISTEMI DI RILEVAMENTO/PREVENZIONE

Un grosso problema nei sistemi in rete è l’accesso ostile (o anche solo “non desiderato”) a macchine e risorse, e questo può avvenire sia **tramite utenti** sia **tramite software**.

- Violazione da parte dell’utente: può essere un **accesso non autorizzato** a una macchina; oppure, se l’utente è autorizzato, può consistere nell’**ottenere privilegi maggiori** (privilege escalation) o nel compiere azioni oltre i limiti concessi.
- Violazione da parte del software: può presentarsi come **virus, worm o Trojan**.

Questi attacchi “riguardano” la sicurezza della rete perché l’accesso può essere ottenuto attraverso una rete, ma non sono limitati agli scenari di rete. Per esempio:

- un utente davanti a un terminale locale può tentare una violazione senza passare da Internet;
- un virus o un Trojans può entrare nel sistema anche da un supporto fisico
- solo il worm come fenomeno è esclusivamente di rete.

Che cos'è un intruder e le 3 classi di Anderson

Una delle due minacce più “pubblicizzate” è l'INTRUDER (l'altra sono i virus) e che spesso viene chiamato hacker o cracker.

Anderson propone tre classi:

1. **Masquerader** (esterno che si finge un utente legittimo): è un individuo non autorizzato che buca i controlli di accesso per sfruttare l'account di un utente reale.
 2. **Misfeasor / Trasgressore** (interno che abusa dei permessi): è un utente legittimo che accede a dati/programmi/risorse per cui non è autorizzato oppure è autorizzato ma abusa dei privilegi.
 3. **Clandestine user / Utente clandestino** (controllo “di supervisione” per eludere audit e controlli): è qualcuno che assume un controllo di livello superiore e lo usa per eludere auditing e controlli di accesso, o addirittura per sopprimere la raccolta di audit (audit sono i controlli per monitorare sicurezza in sistemi IT).
- il masquerader è probabilmente un outsider;
 - il trasgressore è generalmente un insider;
 - l'utente clandestino può essere sia esterno sia interno.

Gli attacchi degli intrusi coprono uno spettro: da intrusi “**benigni**” (esplorazione) fino a intrusi “**gravi**” (lettura di dati privilegiati, modifiche non autorizzate, interruzioni del sistema).

Modelli di comportamento: hacker e criminali, e perché serve difesa a più livelli

Le tecniche e comportamenti degli intrusi cambiano continuamente: sfruttano debolezze appena scoperte e provano a eludere rilevamento e contromisure. Però, nonostante l'evoluzione, spesso seguono pattern riconoscibili, diversi dagli utenti normali. Da qui l'idea di descrivere tre esempi di “modelli” (in queste pagine ne vengono sviluppati due: hacker e criminali).

a) **Hacker** “tradizionali”

Sono descritti come persone che agiscono per gusto o status, dentro una comunità “meritocratica” dove lo status dipende dalla competenza. Per questo tendono a:

- cercare bersagli opportunistici;
- e poi condividere informazioni con altri.

Anche se alcuni intrusi possono essere relativamente “benigni”, non è possibile saperlo a priori: quindi anche sistemi senza risorse “super sensibili” hanno motivi per controllare il problema. IDS e IPS sono sistemi pensati per contrastare questa minaccia “hacker”, e come ulteriori misure l'idea di limitare accessi remoti a IP specifici e/o usare VPN.

I **CERT** (squadre di risposta a emergenze informatiche): raccolgono info su vulnerabilità e le diffondono ai gestori dei sistemi. Ma dato che anche gli hacker leggono questi report, gli admin devono applicare rapidamente le patch. Il testo evidenzia una tensione reale: la complessità dei sistemi e la frequenza delle patch rendono difficile stare al passo senza aggiornamenti automatizzati, che però a loro volta possono creare incompatibilità. Ecco perché si ribadisce la necessità di più livelli di difesa (non una singola misura risolutiva).

b) **Criminali** (hacker organizzati)

Qui il tono cambia: i gruppi organizzati sono una minaccia diffusa e comune per sistemi Internet. Possono essere collegati a aziende o governi, ma spesso sono bande “vagamente affiliate”, e vengono citati contesti come forum clandestini dove scambiano dati e coordinano attacchi.

L'esempio tipico è l'obiettivo e-commerce: ottenere accesso (fino a root) per rubare informazioni preziose, soprattutto dati di carte di credito.

- i numeri rubati vengono usati per acquistare beni costosi e poi pubblicati su siti di “carding” (così altri li riutilizzano);

- questo oscura i pattern di utilizzo e rende le indagini più complesse;
- a differenza degli hacker opportunistici, i criminali hanno spesso obiettivi specifici (o classi di obiettivi);
- una volta entrati, agiscono molto rapidamente, raccolgono più dati possibile e “escono”.

Nota importante: IDS/IPS possono essere usati anche qui, ma potrebbero risultare meno efficaci proprio perché l'attacco è “mordi e fuggi”. Come contromisure mirate agli e-commerce, la pagina raccomanda:

- crittografia del database per i dati sensibili (specie carte di credito);
- se l'e-commerce è ospitato da terzi, usare un server dedicato (non condiviso) e monitorare da vicino i servizi di sicurezza del fornitore.

ATTACCHI INTERNI

Gli attacchi interni (insider attacks) come tra i più difficili da rilevare e prevenire, perché i dipendenti hanno già accesso e soprattutto conoscenza della struttura e del contenuto dei database aziendali. Le motivazioni possono essere vendetta o senso di “diritto” (entitlement).

Anche se IDS/IPS possono aiutare, spesso hanno priorità su misure più dirette:

1. Principio del **privilegio minimo**: ciascuno deve accedere solo a ciò che serve davvero per lavorare.
2. **Logging** per sapere quali utenti accedono e quali comandi inseriscono.
3. **Proteggere risorse sensibili** con autenticazione forte.
4. Quando finisce il rapporto di lavoro, **rimuovere computer e accesso di rete del dipendente**.
5. Al momento del licenziamento, **creare un'immagine speculare del disco** prima di riassegnarlo: può servire come prova se informazioni aziendali compaiono presso un concorrente.

Tecniche Avanzate di Malware e Metodi di Evasione

Oltre alle categorie classiche di malware come virus, worm, spyware e RAT (Remote Access Trojan), i software malevoli moderni — inclusi i **ransomware** che cifrano i file per estorcere riscatti — utilizzano strategie sofisticate per aggirare le difese aziendali come Antivirus, Firewall ed EDR (Endpoint Detection and Response).

Per eludere i sistemi di sicurezza, i **malware agiscono principalmente su due fronti: l'evasione dell'analisi statica e il blocco dell'analisi dinamica**.

Evasione dell'Analisi Statica: l'analisi statica (come le regole YARA) cerca di identificare il malware senza eseguirlo, analizzando il codice e cercandone la “firma” o pattern sospetti. I malware bypassano questo controllo tramite:

- **Offuscamento con OLLVM:** L'uso di compilatori come OLLVM permette di generare un codice binario diverso a ogni compilazione, rendendo impossibile l'identificazione tramite firme fisse.
- **PE Packers e Crypters:** Queste tecniche mantengono il malware cifrato all'interno di un “involucro” (packer) e lo decifrano solo in fase di runtime (esecuzione), rendendo il codice malevolo invisibile ai controlli statici.
- **Motori Metamorfici e Polimorfici:** Questi motori cambiano costantemente la struttura del malware o del “crypter” stesso, alterando continuamente la firma digitale per sfuggire ai database degli antivirus.

Evasione dell'Analisi Dinamica (Antivirus Termination) L'analisi dinamica osserva il comportamento del malware durante l'esecuzione. La tecnica più aggressiva per bypassare questo controllo è l'**Antivirus Termination**:

- Il malware tenta di “uccidere” o terminare i processi dei software di sicurezza attivi sulla macchina.
- Sui sistemi Windows, questo viene spesso realizzato invocando funzioni di basso livello come **ZwTerminateProcess**, che permette di terminare forzatamente ogni tipo di processo, inclusi quelli degli antivirus, neutralizzando così la difesa in tempo reale.

IDS e IPS:

IDS (Intrusion Detection System)

L'IDS viene definito come software che automatizza il monitoraggio degli eventi su un sistema o su una rete e poi analizza ciò che osserva per riconoscere segnali di incidenti: cioè violazioni o minacce imminenti di violazioni di policy (policy di sicurezza, policy di utilizzo accettabile, pratiche standard).

- **Identificazione** e analisi dei possibili incidenti: l'IDS non “blocca” per definizione, ma **deve prima capire se c'è un evento anomalo o malevolo**.
- **Logging/Recording** degli eventi osservati: i log possono essere tenuti localmente, oppure inviati a sistemi separati come server di logging centralizzati, soluzioni SIEM (Security Information and Event Management) e sistemi di gestione aziendale. L'idea è: non basta vedere l'evento “in diretta”, serve anche poterlo ricostruire, correlare e auditare.
- **Notifica** (alert) **agli amministratori di sicurezza**: Alcuni canali concreti (e-mail, pagine/pager, messaggi sull'interfaccia dell'IDPS, trappole SNMP, syslog, script/programmi definiti dall'utente). In pratica, l'IDS deve far arrivare il segnale a chi può intervenire.
- **Reportistica**: l'IDS produce report per riassumere gli eventi monitorati o dare dettagli su eventi di particolare interesse. Inoltre nota un aspetto pratico: spesso la notifica contiene solo info “di base”, e l'amministratore deve entrare nella console/IDPS per vedere i dettagli completi.

IPS (Intrusion Prevention System)

L'IPS è un sistema che ha tutte le funzionalità dell'IDS ma in più può tentare di fermare i possibili incidenti. Il focus quindi è l'azione attiva: non solo vedere e avvisare, ma interrompere o depotenziare l'attacco.

L'obiettivo dell'IPS può essere scomposto in tre famiglie di intervento:

1. **Fermare l'attacco stesso: interrompere connessioni o sessioni** usate per l'attacco; **bloccare l'accesso alla destinazione** (o ad altre probabili destinazioni) basandosi su attributi dell'aggressore (account, IP, ecc.); fino a bloccare tutti gli accessi a host/servizio/applicazione bersaglio.
2. **Cambiare l'ambiente di sicurezza**: l'IPS può anche “spostare” le difese **riconfigurando altri dispositivi (firewall, router, switch) per bloccare l'attaccante**; alcuni IPS possono portare all'applicazione di patch su un host se vengono rilevate vulnerabilità.
3. **Modificare il contenuto dell'attacco**: alcune tecnologie IPS possono **rimuovere o sostituire parti dannose** (esempio semplice: togliere un allegato infetto da una mail). Un esempio più complesso è l'**IPS che fa da proxy** e “normalizza” le richieste in ingresso (riconfeziona i payload, scarta info di header): così certe tecniche di attacco possono “rompersi” durante la normalizzazione.

IDPS (Intrusion Detection and Prevention System)

L>IDPS è la combinazione di IDS e IPS: quindi **difesa attiva e in tempo reale**. L'idea non è solo “bloccare attacchi”, ma anche migliorare governance e controllo:

1. **Identificare problemi di policy di sicurezza**: l>IDPS può fungere da controllo qualità dell'implementazione delle policy (esempio: **individuare duplicazioni di regole firewall**; oppure **segnalare traffico che avrebbe dovuto essere bloccato dal firewall** ma passa per un errore di configurazione).
2. **Documentare le minacce**: registra info sulle minacce rilevate (frequenza e caratteristiche degli attacchi). Questo è utile sia per **scegliere misure di sicurezza adeguate**, sia per “educare” il management su quali minacce l'organizzazione sta davvero affrontando.
3. **Scoraggiare violazioni di policy**: se le persone sanno che le azioni sono monitorate, possono evitare violazioni perché aumenta il rischio di essere scoperti.

Quindi: un IDPS non è solo tecnologia “anti-hacker”; è anche uno strumento di controllo, evidenza e deterrenza.

Dal punto di vista tecnico, un IDPS funziona perché confronta ciò che osserva con criteri di rilevamento. Alcuni criteri sono basati su **firme**: se **riconosce una sequenza di byte** o un **pattern tipico di un exploit noto**, scatta l'allarme. Altri criteri sono **basati sull'anomalia**: si costruisce un'idea di “normalità” e **tutto ciò che devia troppo viene marcato come sospetto**. Altri ancora si basano su **analisi di protocollo**: il sistema controlla se **una comunicazione segue il protocollo in modo coerente**, e segnala deviazioni che spesso sono sintomo di manipolazioni o attacchi. Nella pratica i prodotti moderni combinano questi approcci per coprire sia minacce note sia comportamenti strani non ancora catalogati.

Un elemento che spesso distingue l'IDPS è il tema della messa a punto. Nessun sistema di rilevamento è perfetto: esistono **falsi positivi** (allarmi su attività legittime) e **falsi negativi** (attacchi reali non rilevati). Se rendi il sistema più "sensibile", vedrai più attacchi ma anche più rumore; se lo rendi più "selettivo", riduci gli allarmi ma rischi di perdere incidenti. Per questo **un IDPS non è qualcosa che si installa e basta**: va **calibrato** sull'**ambiente**, sulle **applicazioni**, sugli **orari** e sui flussi di lavoro reali, e va **rivisto nel tempo**. È una tecnologia che funziona bene solo se esiste un minimo di processo: qualcuno deve guardare gli alert, capire cosa è rilevante, aggiornare regole e soglie, e trasformare gli eventi in azioni concrete (hardening, patching, correzione di configurazioni).

INTEGRAZIONE IDPS

l'IDPS diventa davvero efficace quando è integrato nel resto dell'ecosistema di sicurezza. Da solo vede "pezzi" della realtà; collegato a log centralizzati e sistemi di correlazione (tipicamente un SIEM) **può unire i segnali, ricostruire una catena di eventi** e rendere più semplice distinguere un falso allarme da un attacco in corso. In sintesi, l'IDPS è un guardiano che osserva e segnala (e talvolta blocca), ma il suo valore massimo emerge quando è inserito in una strategia più ampia, con tuning, manutenzione e gestione operativa.

FIREWALL

Firewall: è un dispositivo/sistema che **controlla il flusso di traffico** tra reti con livelli di sicurezza diversi. Ovvero una rete interna fidata e una esterna non fidata.

Un firewall fatto bene punta a **3 obiettivi chiave**.

1. Non devono esistere scorciatoie. **Se qualcuno può entrare nella LAN senza passare dal firewall non è un firewall valido**. Quindi si cerca di fare in modo che tutto il traffico in entrata e in uscita passi davvero da quel punto.
2. **Passa solo ciò che è autorizzato** dalla politica di sicurezza
3. Firewall stesso **deve essere duro da bucare**, deve avere una **configurazione minimale**.

Politiche di accesso

Il firewall è un **esecutore di regole**. La politica dice quali traffici sono ammessi: **quali indirizzi, quali protocolli, quali applicazioni, quali contenuti**.

Le regole possono basarsi su diversi tipi di criteri, quello più classico è quello di rete/trasporto: IP sorgente/IP destinazione, porte, direzione(in/out), protocollo e simili.

Possiamo salire di livello e **filtrare in base al protocollo applicativo**(es: controllare SMTP per spam, controllare HTTP per limitare siti o tipi di richieste (quando possibile)).

Un'altra famiglia di regole usa l'**identità utente**: invece di dire chiunque da questo IP dici solo utenti autenticati o gruppi specifici.

Regole basate sul comportamento della rete: consenti solo in orario di lavoro, blocca se vedo troppe richieste in poco tempo.

Benefici del firewall

Diventa un punto di strozzatura, se prima avevi tanti host esposti e regole sparse, **con il firewall hai un posto unico dove far rispettare la policy**. Può **mitigare varie forme di spoofing e attacchi routing**.

Un altro beneficio molto importante è il **Logging**. Un firewall è un punto privilegiato per registrare eventi di sicurezza (senza Log seri spesso non riesci a capire cosa ti sta succedendo).

Schermatura: un firewall può **schermare parti della rete interna per renderle meno visibili dall'esterno**, può bloccare servizi specifici, può proteggere risorse interne e può aiutare a prevenire l'uscita di dati se la policy è fatta bene.

Cosa non fa un firewall

Un firewall non ti salva se l'attacco lo aggira: ad esempio un canale alternativo, un accesso diretto, una VPN non controllata un wifi mal configurato.

Non ti protegge bene dalle **minacce interne**.

Non è una di difesa efficiente contro bug nuovi e sconosciuti nei protocolli o nelle applicazioni. Nella realtà operativa le **regole sono difficili da mantenere** perché devo trovare un compromesso sul bloccare tutto o lasci lavorare.
Un firewall può anche rallentare la rete soprattutto se fa ispezione profonda o gestisce tanti flussi.

Tipi di Firewall :

Packet filtered firewall: guarda ogni pacchetto IP in entrata e in uscita e lo confora con un elenco di regole.

Le regole sono basate su campi tipo: IP sorgente/destinazione, Porte TCP/UDP sorgente e destinazione. Protocollo (TCP UDP ICMP ecc...), interfaccia di ingresso/uscita dei firewall

Come funziona? Se un pacchetto combacia con una regola, la regola dice “passa” o “blocca” se non combacia con nulla entra in gioco la politica di default:

Default deny : se non è esplicitamente consentito, è vietato.

Default permit: se non è esplicitamente vietato è consentito

Vantaggi : è **semplice, trasparente** per l’utente, e spesso **molto veloce** perché non deve capire davvero l’applicazione.

Svantaggi: se non guarda i dati applicativi, **non può fermare bene gli exploit che stanno dentro HTTPS/SMTP, spesso logging meno ricco e soprattutto scrivere regole buone non è banale.**

Tre classici attacchi contro il packet filter:

Spoofing dell’IP Sorgente: l’attaccante dell’esterno mette come IP sorgente un IP interno fidato sperando che il firewall o qualche host si fidi del solo indirizzo.

Come evitarlo : se un pacchetto arriva dall’interfaccia esterna e dichiara un IP sorgente interno, lo scarti.

Source routing : l’attaccante prova a specificare il percorso del pacchetto per aggirare controlli o farlo arrivare in modo anomalo. Contromisura tipica: scartare pacchetti che usano opzioni di source routing.

Tiny fragment attack: qui l’attaccante frammenta il pacchetto IP in pezzi minuscoli in modo che le informazioni importanti dell’header TCP (ad esempio le porte) finiscono in un frammento successivo. Se il Firewall decide solo guardando il primo frammento, rischia di non vedere abbastanza per applicare correttamente la regola e lascia passare frammenti che poi ricompongono un traffico che avrebbe dovuto bloccare.

Cosa fare ? : è imporre che il primo frammento contenga almeno una quantità minima dell’header di trasporto e/o fare tracing in modo che se rifiuti il primo frammento, rifiuti anche tutti gli altri frammenti associati.

Perché non basta sempre un Packet filter classico

Un packet filter tradizionale guarda un pacchetto alla volta e decide “passa/scarta” in base a campi tipo IP sorgente/destinazione, porte, protocollo. Il problema è che spesso, in rete, un singolo pacchetto non ti dice abbastanza (mancanza di contesto).

Il contesto tipico è la connessione TCP.

Es: pensando a SMTP il server sta sulla porta 25, ma il client non usa una porta fissa: usa una porta alta scelta al volo. (Tra 1024 e 65535). Quindi una comunicazione reale funziona così: il client apre una connessione da una porta alta verso una porta nota del server (25 in questo caso) . Poi quando il server risponde le risposte tornano verso la porta alta del client.

Se utilizzassimo un firewall che ragiona pacchetto per pacchetto senza contesto potremmo trovarci davanti a questo dilemma: voglio permettere al client interno di aprire connessione verso fuori (OK). Ma come faccio a permettere le risposte in ingresso verso porte alte, senza aprire una voragine

Per risolvere questa problematica nasce lo STATEFUL INSPECTION FIREWALL che praticamente continua a guardare gli stessi campi di un packet filter ma in più mantiene memoria delle connessioni attive.

In pratica costruisce una Tabella delle connessioni TCP che sono state aperte dall'interno verso l'esterno : per ogni connessione registra un profilo composto da Ip interni/esterni, porte, stato della connessione, ecc.)

Quando arriva traffico dell'esterno verso una porta interna "alta" il firewall si chiede: questo pacchetto è una risposta con una connessione che ho visto partire dall'interno ? Se sì lo lascia passare. Se no, lo Blocca.

Per un approccio ancora più forte si usa **L'application-level Gateway** : il firewall non si limita a controllare IP/porte e lo stato TCP : agisce proprio come un intermediario dell'applicazione.

Invece di avere una connessione diretta client ↔ server succede questo:

Il client parla con il proxy(il firewall /gateway) usando l'applicazione (FTP, HTTP, SMTP). Il proxy, dopo aver autenticato l'utente e applicato le regole apre lui una seconda connessione verso il server estero.

Il proxy inoltra i dati tra le due connessioni.

Facendo così il Gateway può anche dire: questo protocollo lo supporto solo in certe funzionalità, tutto il resto lo nego. E può loggare e controllare in maniera molto più ricca perchè vede il traffico a livello applicativo.

Svantaggio: costo elevato nel senso di overhead perchè ci sono due connessioni e perchè il Gateway deve ispezionare e gestire tutto il traffico applicativo.

Una via di mezzo è il **Circuit-Level-Gateway**: anche lui rompe la connessione end-to-end e crea due connessioni TCP (una interna e una esterna), però non legge il contenuto applicativo, inoltra solitamente i segmenti TCP così come sono solo dopo aver deciso se quella connessione è permessa.

Quando si usa: è utile quando vuoi controllare chi può aprire connessioni e verso dove.

Uso tipico è quando ti fidi degli utenti interni : magari controlla molto bene l'ingresso ma sull'uscita non vuoi overhead enorme.

Dove "vive" un firewall e cosa significa bastion host

Quando implementi un firewall, spesso lo metti su una macchina dedicata o su apparati di rete. In molte architetture compare il concetto di Bastion HOST : ovvero un computer pensato per essere esposto e resistere meglio (pochi servizi essenziali, accesso limitato, software modulare e controllabile), spesso ospita un proxy applicativo o un proxy di circuito.

Oltre al firewall di confine esistono anche firewall su singolo HOST.

Host-based firewall: è un modulo software su server. Ha senso perchè puoi fare regole su misura su quella macchina e soprattutto perchè funziona anche contro il traffico interno.

Personal firewall: versione più leggera pensata per PC/portatili. Il compito principale è impedire accessi remoti non autorizzati e spesso controlla anche il traffico in uscita per individuare comportamento sospetto.

DMZ la zona "sacrificabile ma controllata"

La parte fondamentale di una architettura è dove metto i server pubblici. Solitamente non voglio che un server esposto stia dentro la rete interna come un normale PC. Per questo si crea una DMZ una rete intermedia tra interno e esterno.

Il modello tipico è :

1. Firewall esterno: gestisce i servizi che devono essere raggiungibili da internet (es. permetto traffico verso il web server in DMZ)
2. Firewall interno : protegge la rete aziendale vera con regole molto più rigide.

Il firewall interno ha tre scopi principali :

1. filtra più duramente per proteggere server e workstation interne.
2. Protegge la rete interna anche da eventuali compromissioni della DMZ
3. Può separare ulteriormente la rete interna in zone (segmentazione)

VPN

Una VPN serve quando vuoi collegare sedi o utenti remoti usando una rete non sicura (internet) ma mantenendo confidenzialità e autenticazione. Spesso la tecnologia usata è IPsec.

Ma dove si mette IPsec rispetto al firewall?

1. Se cifri il traffico prima che passi nel firewall (cioè il firewall vede solo dati cifrati), il firewall non riesce più a fare bene molte ispezioni/controlli applicativi e perfino alcune logiche di filtro e logging diventano più limitate.
2. Se metti IPsec sul firewall stesso, hai un punto unico dove gestisci sia protezione sia policy/controlli.
3. Metterlo sul router esterno è possibile, ma spesso il router è meno robusto di un firewall dedicato.

Firewall distribuiti

Un'evoluzione moderna è l'idea di Firewall distribuiti: non solo il perimetro ma anche regole su server e workstation(host-based) tutti gestiti con un controllo centrale. Il che può aiutare molto contro minacce interne e permette policy più specifiche per applicazioni/machine.

Nel contesto di distribuito puoi avere anche DMZ interne e esterne con logiche più specifiche e diventa molto importante il monitoraggio centralizzato.

Le tre architetture riassuntive (screened host / screened subnet) dette bene

Abbiamo tre configurazioni classiche:

1. Screened Host, single-homed bastion: il bastion host ha una sola interfaccia e sta davanti alla rete interna. Il router verso internet lascia parlare praticamente solo con il bastion. L'idea è : chi vuole entrare passa dal bastion.
2. Screened Host, dual-homed bastion: il bastion host ha due interfacce (una verso internet e una verso LAN) . Tutto il traffico tra esterno e interno deve passare fisicamente dal bastion-> più controllo.
3. Screened subnet(DMZ): hai una subnet schermata tra firewall esterno e interno (la DMZ) . È una configurazione molto comune per server pubblici: web/mail/dns stanno in DMZ, l'interno resta più protetto